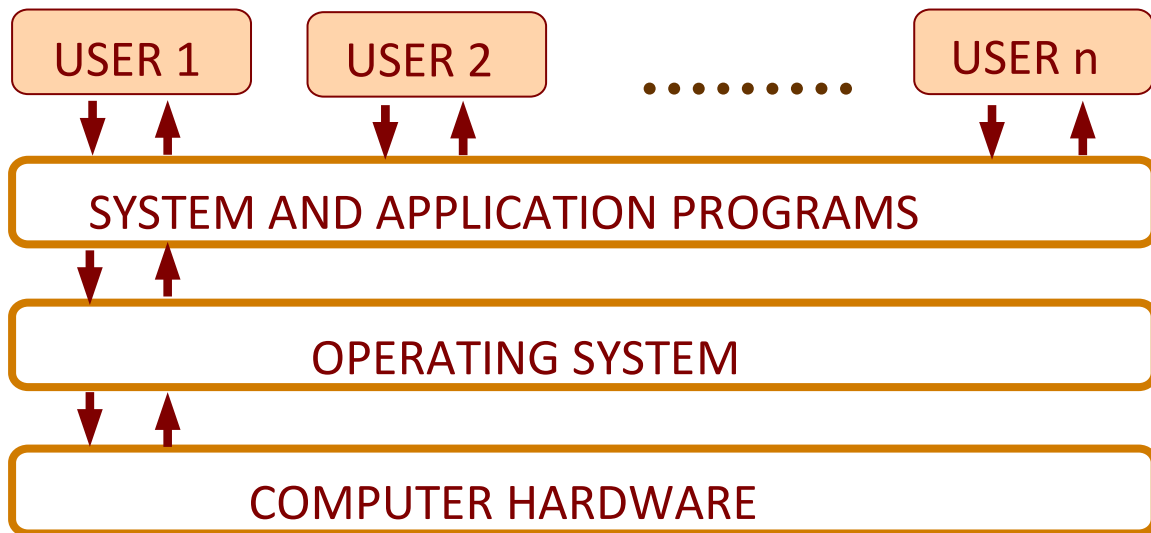


An operating system (OS) is a software program that manages and coordinates the resources of a computer. It acts as an intermediary between the computer hardware and the software applications that run on it, providing a platform for these applications to execute.



Two primary goal of OS:

1. Convenience/user friendly(windows)
2. Throughput/Efficiency(Linux)

The operating system is responsible for tasks such as:

Memory management: Allocating and managing the computer's memory resources.

Processor management: Scheduling and controlling the execution of processes.

Input/output management: Coordinating and controlling the transfer of data to and from the computer's hardware devices.

File management: Organizing and managing files and directories on the computer's storage.

Security: Protecting the computer and its resources from unauthorized access.

Some popular operating systems include Windows, macOS, Linux, Unix, and Chrome OS. Each operating system has its own set of features and functionalities, and choosing the right operating system depends on the specific needs and requirements of the user.

There are two main views of an operating system: the user view and the system view.

User view: The user view of an operating system is focused on the services and interfaces that the operating system provides to the users. This view includes the graphical user interface (GUI), file management system, and application programming interfaces (APIs). The user view is concerned with how the operating system makes it easier for users to interact with the computer and perform various tasks.

System view: The system view of an operating system is focused on the underlying mechanics of the operating system and how it manages the hardware and software resources of the computer. This view includes the process management, memory management, device management, and security features of

the operating system. The system view is concerned with how the operating system works to coordinate and control the different components of the computer.

Both the user view and system view are important for understanding the role and functions of an operating system. The user view provides a perspective on how the operating system can be used to perform various tasks, while the system view provides a deeper understanding of the internal workings of the operating system.

The evolution of operating systems has been driven by the advancements in computer hardware and the increasing demands for better performance and functionality. Here are some key milestones in the evolution of operating systems:

Mainframe operating systems (1960s-1970s): The first mainframe computers used simple batch processing operating systems that processed jobs in a queue. These early operating systems were designed to maximize the utilization of the mainframe's processing power.

Time-sharing operating systems (1960s-1970s): With the development of multi-user systems, time-sharing operating systems emerged, allowing multiple users to access the mainframe simultaneously.

Personal computer operating systems (1980s-1990s): The introduction of personal computers brought the development of personal computer operating systems such as MS-DOS, Apple's Macintosh System Software, and the early versions of Windows.

Modern desktop operating systems (1990s-present): The 1990s and 2000s saw the development of modern desktop operating systems, such as Windows 95, Windows XP, Windows 7, macOS, and Linux. These operating systems provided a graphical user interface, improved memory management, and improved multimedia support.

Mobile operating systems (2000s-present): With the advent of smartphones and tablets, mobile operating systems such as iOS, Android, and Windows Mobile emerged, providing a new platform for mobile computing.

Cloud operating systems (2010s-present): With the growth of cloud computing, new operating systems designed for cloud environments have emerged, such as Google's Chrome OS and Amazon's AWS.

Each generation of operating systems has brought new features, improvements, and capabilities to the field, and the evolution of operating systems is ongoing, driven by the demands of new technologies and the needs of users.

Different Types of Operating System (OS) or evolution

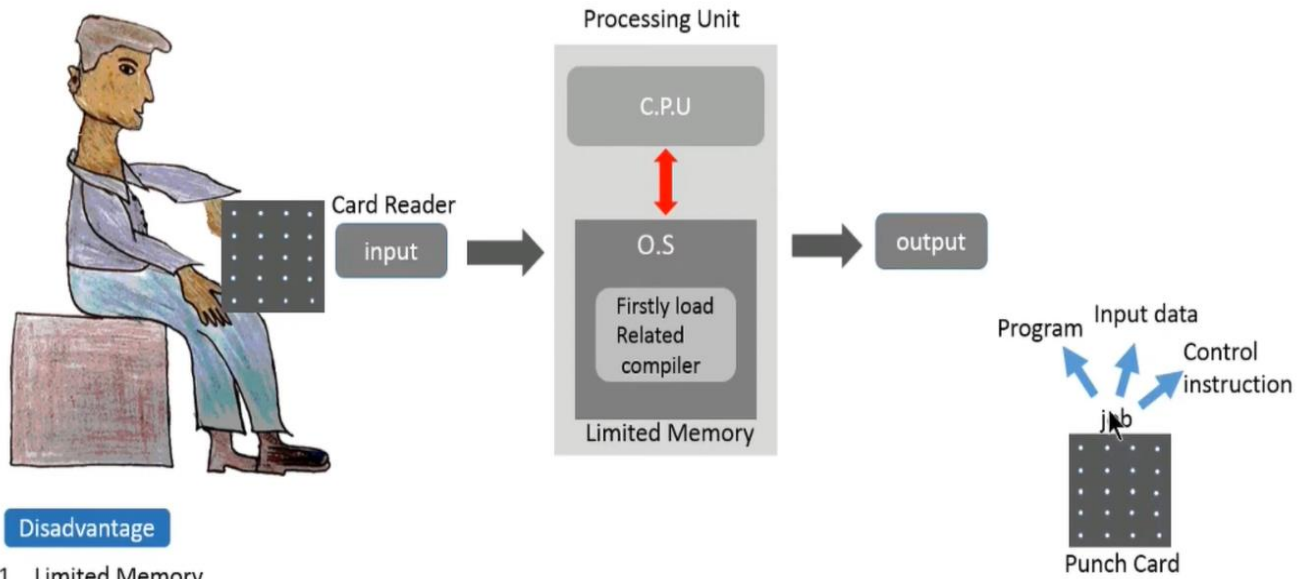
1. Simple Batch Operating System

In starting main frame computer

- Common I/P and O/P devices were punch card and tape drive
- User prepare a job which consists of program , I/P data and control instruction
- I/P job is given in the form of punch card through card reader and result is also generated in the form of punch card after processing
- So OS was very simple, always present in the memory major task is to transfer the control from one job to another.

Job

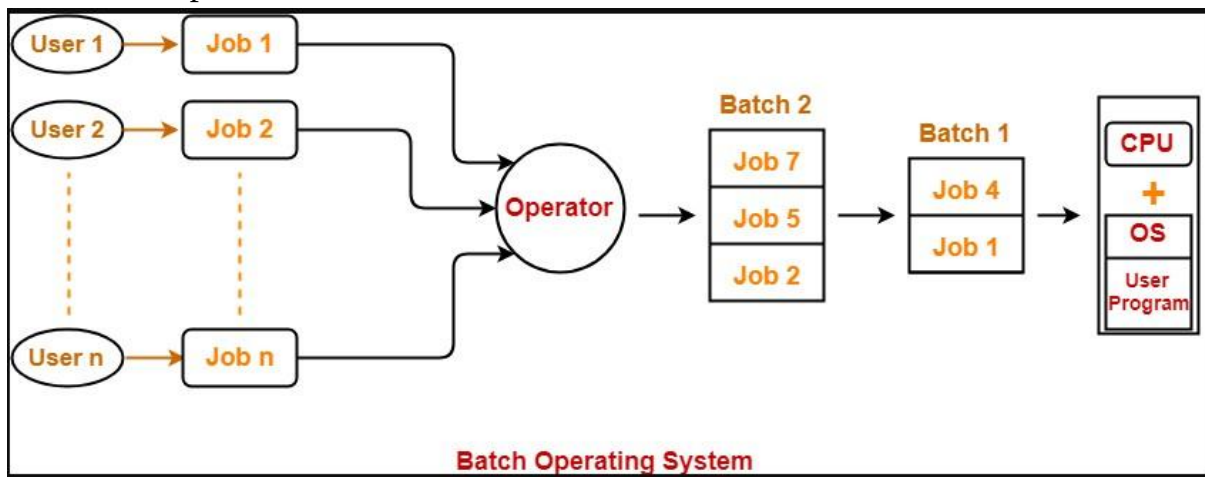
Program
I/P data
Control instruction



1. Limited Memory
2. CPU idle for long time
3. Lack of interaction b/w user and his job/process

Batch processing:

- Jobs with similar needs are batched together and executed through the processor as a group.
- Operators sort jobs as a deck of punch card in to batch with similar needs
- Example: FORTAN batch, COBOL batch.



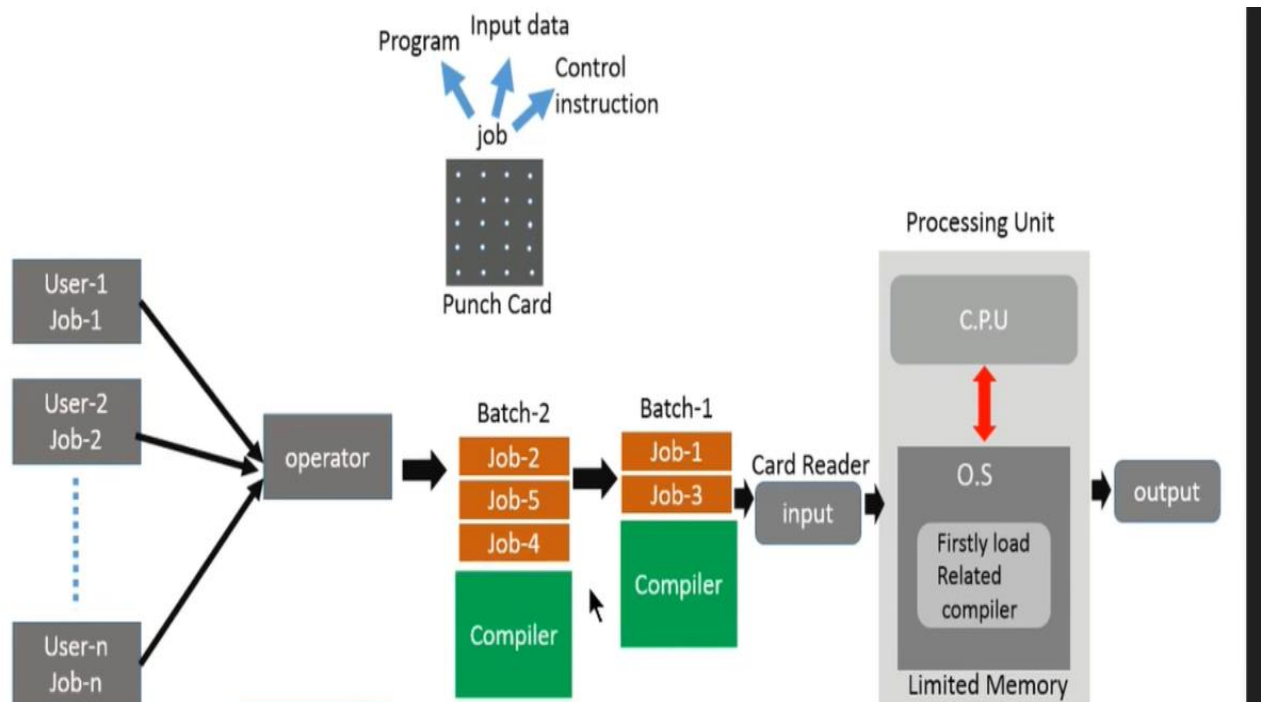
Advantage:

- In a batch job executes one after another saving time from activities like loading compiler

- During a batch execution no manual interaction is needed

Disadvantage:

- Memory limitation
- Interaction of I/P and O/P devices are directly with CPU.
- Most of the time CPU remain Idle



Disadvantage

1. Limited Memory
2. CPU idle for long time
3. Lack of interaction b/w user and his job/process

Spooling (simultaneous peripheral operations online)

- I/P and O/P devices are relatively slow in compare to CPU(Digital)
- Spooling is an I/O management or buffer management technique that allows the data of the input/output processes to be temporarily stored in the secondary memory which will be executed by the CPU.
- These data will be stored in the secondary memory until the system or a program requests the data for its execution.
- In spooling data is stored first in to the disk and then CPU interact with disk(digital) via main memory
- Spooling is capable of overlapping I/O operations for one job with CPU operation of other job.
- Previously, when the existence of an operating system came, it used to give input to the CPU, and then the CPU executes that particular data and then accordingly produces the output. • Now, in this process 'there exist a problem where the number of processes is more, and as we already know the I/O operations take relatively more time than the CPU takes for the execution of the same.
- After the completion of one process, the next process starts its execution which is one of the worst conditions in an OS as its main goal is to provide maximum CPU utilization, and in this process, the CPU is idle for most of the time. Here, the existence of spooling came to play.
- I/O devices like keyboards, mouse, printers, etc., are slower devices when compared to CPU and main memory. Therefore, if a peripheral device interacts directly with the CPU for its execution then the overall performance of the system will be bottlenecked. which will eventually also result in bad CPU utilization.
- In spooling the data stored in the secondary memory or buffer is transferred to the main memory (RAM). This helps in proper utilization of the CPU as the CPU directly interacts with the main

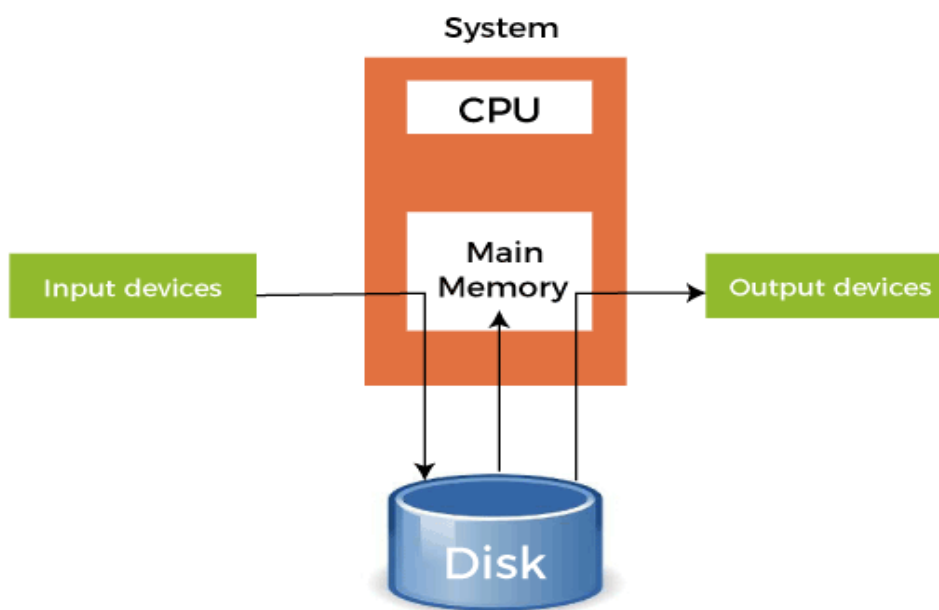
memory rather than the I/O devices which is as fast as the processing of a CPU.

Advantage:

- No interaction of I/P and O/P devices are directly with CPU.
- CPU utilization is more as CPU is busy most of the time.

Disadvantage:

- In starting spooling was uniprogramming.



2. Multiprogramming Operating System

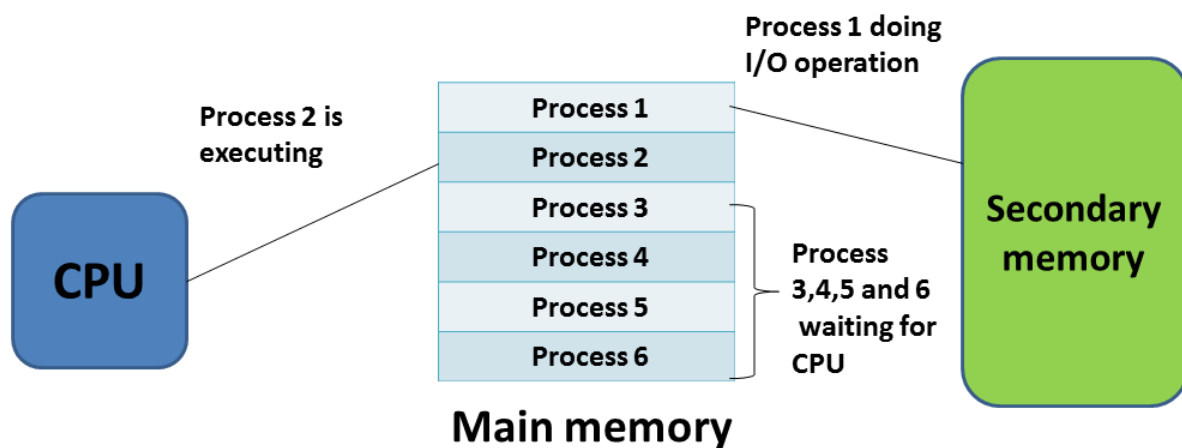
- Maximum CPU utilization
- Multiprogramming means more than one process in main memory which are ready to execute.
- Process generally require CPU time and I/O time, so if running process perform I/O operation or some other event which do not require CPU then instead of sitting idle CPU make context switch and picks some other process and this idea will continue.
- CPU never become idle unless there is no process in ready state in main memory.

Advantage:

- High CPU Utilization
- Less waiting time, response time etc.

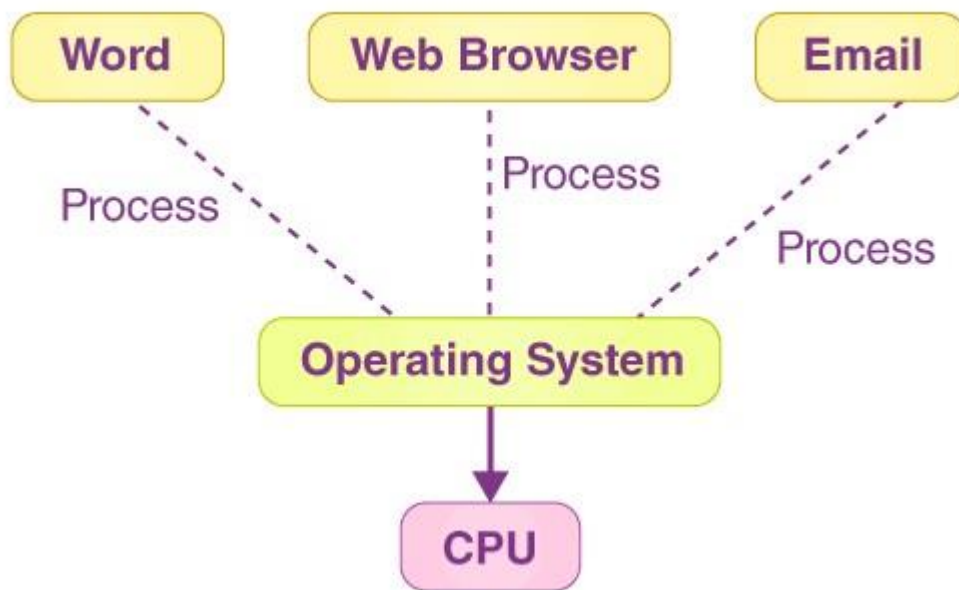
Disadvantage:

- Difficult scheduling.
- Main memory management is required
- Memory fragmentation
- Paging (non –contiguous memory allocation)



3. Multi-tasking/Time-Sharing Operating System/Fair share

- Multi-tasking is multiprogramming with time sharing.
- Only one CPU but switches between process so quickly that it gives an illusion that all process executes at the same time.
- The task in multi-tasking may refer to multiple threads of the same program.
- Main idea is better response time and executing multiple process together.



4. Multiprocessing Operating System

- Two or more CPU with in single computer in close communication sharing the system bus, memory and other I/O devices.
- Different process may run on different CPU, true parallel execution.
- **Symmetric** - one OS control all CPU, each CPU has equal rights.
- **Asymmetric** – Master slave architecture, system takes on one processor and application on other or one CPU will handle all hardware interrupt or I/O devices they are easy to design but less efficient.

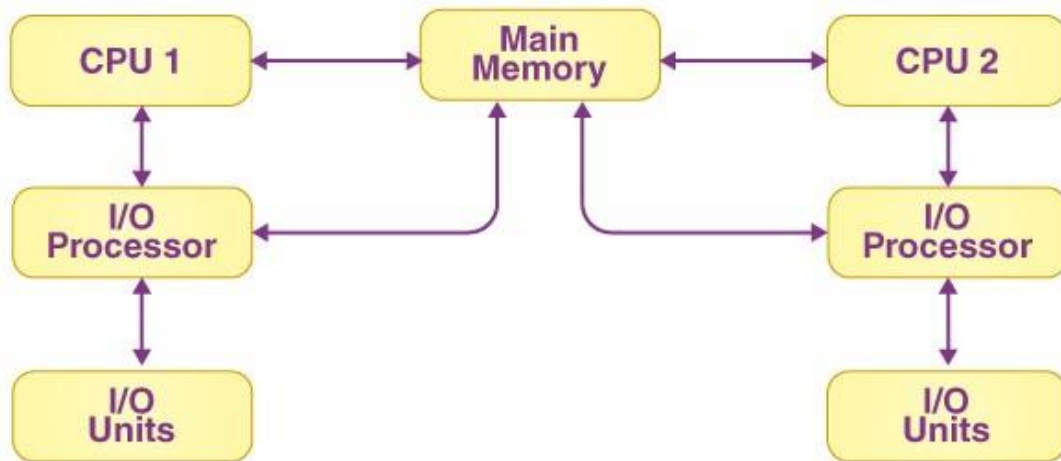
Advantage:

- Increased throughput
- Increased reliability/ graceful degradation
- Cost saving
- Battery efficient

- True parallel processing

Disadvantage:

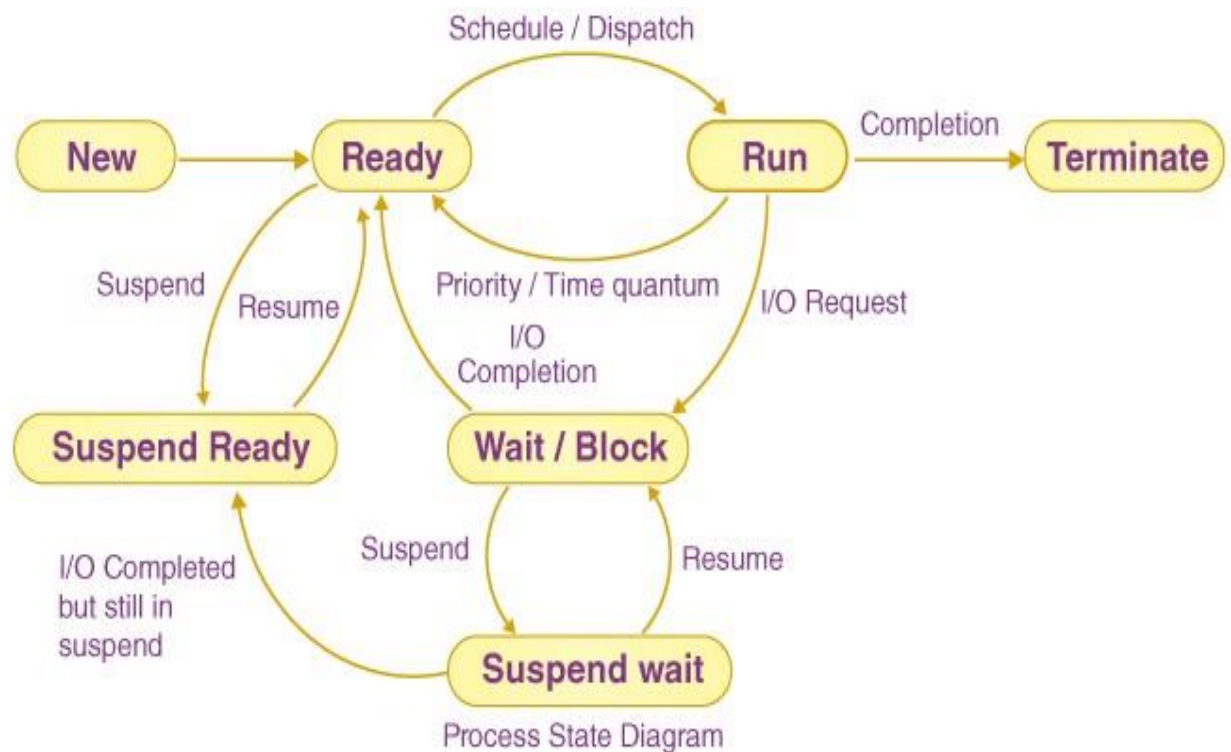
- More complex
- Large main memory



Working of Multiprocessor System

Process states

From start to finish, the process goes through a number of stages. A minimum of five states is required. Even though the process could be in one of these states during execution, the names of the states are not standardised. Throughout its life cycle, each process goes through various stages. They are:



New State

When a program in secondary memory is started for execution, the process is said to be in a new state.

Ready State

After being loaded into the main memory and ready for execution, a process transitions from a new to a ready state. The process will now be in the ready state, waiting for the processor to execute it. Many processes may be in the ready stage in a multiprogramming environment.

Run State

After being allotted the CPU for execution, a process passes from the ready state to the run state.

Concept of pre-emptive and non-pre-emptive

Terminate State

When a process's execution is finished, it goes from the run state to the terminate state. The operating system deletes the process control box (or PCB) after it enters the terminate state.

De allocation happens here

Block or Wait State

If a process requires an Input/Output operation or a blocked resource during execution, it changes from run to block or the wait state.

The process advances to the ready state after the I/O operation is completed or the resource becomes available.

Suspend Ready State

If a process with a higher priority needs to be executed while the main memory is full, the process goes from ready to suspend ready state. Moving a lower-priority process from the ready state to the suspend ready state frees up space in the ready state for a higher-priority process.

Until the main memory becomes available, the process stays in the suspend-ready state. The process is brought to its ready state when the main memory becomes accessible.

Suspend Wait State

If a process with a higher priority needs to be executed while the main memory is full, the process goes from the wait state to the suspend wait state. Moving a lower-priority process from the wait state to the suspend wait state frees up space in the ready state for a higher-priority process.

The process gets moved to the suspend-ready state once the resource becomes accessible. The process is shifted to the ready state once the main memory is available.

Present in Memory	State
Secondary Memory	New state
Main Memory	Ready state
Main Memory	Run state
Main Memory	Wait state
Secondary Memory	Suspend wait state
Secondary Memory	Suspend ready state